

COURSE MANAGEMENT SYSTEM

Project Report

Technologies Used:

HTML, CSS, JavaScript, Python, Django, Django REST Framework, SQLite, Postman

Submitted By:

Aashifa Sahanaj M

Department:

Electronics and Communication Engineering

Academic Year:

2026–2027

1.introduction

The Course Management System is a web-based application developed to manage course information efficiently. The system allows users to add, view, update, delete, and search course details.

The application is developed using HTML, CSS, and JavaScript for the frontend, Python and Django for the backend, Django REST Framework for REST APIs, and SQLite for database management.

The system provides a simple and user-friendly interface for managing course records. REST APIs are used to perform CRUD operations between the frontend and backend.

2. Problem Statement

Managing course information manually can be time-consuming and may lead to errors in maintaining records.

The Course Management System is developed to provide a digital solution for storing and managing course details. It reduces manual work and makes course information easier to access, update, and delete.

The system stores important details such as Course ID, Course Name, Department, Instructor, and Duration.

3. Objectives

The main objectives of the Course Management System are:

- To develop a simple web-based course management application.
- To store course information in a database.
- To provide CRUD operations for course records.
- To allow users to search for courses.
- To provide input validation.
- To develop REST APIs using Django REST Framework.
- To test the APIs using Postman.
- To provide a simple and user-friendly interface.
- To maintain course information efficiently.

4. Technologies Used

Frontend

HTML:

Used to create the structure of the web page.

CSS:

Used to design and style the application interface.

JavaScript:

Used to provide dynamic functionality such as adding, editing, deleting, searching, and loading course data.

Backend

Python:

Used as the programming language for backend development.

Django:

Used as the web framework for developing the backend application.

Django REST Framework:

Used to create REST APIs for performing CRUD operations.

Database

SQLite:

Used to store course information.

Testing

Postman:

Used to test REST API operations such as GET, POST, PATCH, and DELETE.

Development Tool

Visual Studio Code:

Used for writing and managing the project source code.

5. System Architecture

The system follows a simple three-layer architecture.

1. Frontend Layer

The frontend is developed using HTML, CSS, and JavaScript. It provides the user interface for adding, viewing, editing, deleting, and searching courses.

2. Backend Layer

Django and Django REST Framework handle the application logic and REST API requests.

3. Database Layer

SQLite stores the course information permanently.

Data Flow

User → Frontend → REST API → Django Backend → SQLite Database

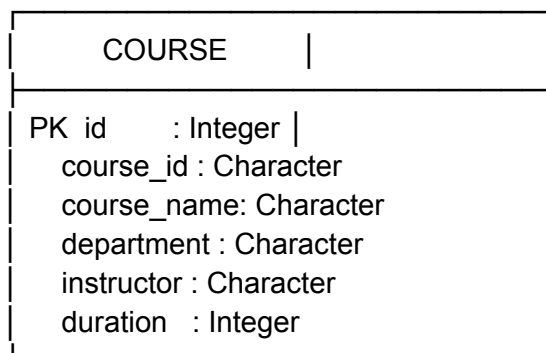
The response is then returned from the database through the backend API to the frontend.

6.Database Design

The system uses an SQLite database to store course information

Field	Data type	Description
Id	Integer	Primary key
Course id	Character	Unique course identifier
Course name	Character	Name of the course
Department	Character	Department offering the course
Instructor	Character	Course instructor
Duration	Integer	Course duration in months

ER Diagram



7. Module Description

7.1 Add Course Module

This module allows the user to add a new course by entering:

- Course ID
- Course Name
- Department
- Instructor
- Duration

The entered information is sent to the backend using a POST request and stored in the SQLite database.

7.2 View Course Module

This module displays all available course records in a table.

The frontend retrieves the course information using the GET API.

7.3 Edit Course Module

The edit option allows the user to update the course name of an existing course.

The update operation is performed using the PATCH API.

7.4 Delete Course Module

The delete option allows the user to remove an existing course record.

The DELETE API is used for this operation.

7.5 Search Course Module

The search feature allows users to search/filter course records based on the information displayed in the course table.

7.6 Validation Module

The system validates course information before storing it.

For example, the course duration must be greater than zero.

Duplicate Course IDs are also prevented using a unique database constraint.

8. Crude Operations

CRUD stands for:

- Create
- Read
- Update
- Delete

The Course Management System implements all four CRUD operations.

Create

A new course is added using the POST request.

Read

Existing courses are retrieved using the GET request.

Update

Course information is updated using the PATCH request.

Delete

An existing course is removed using the DELETE request.

CRUD operations make the application useful for complete course record management.

9. Rest API Documentation

The application provides REST API endpoints using Django REST Framework.

Method	End point	Purpose
GET	/api/courses/	View all courses
POST	/api/courses/	Add a new course
GET	/api/courses{id}/	View a specific course
PATCH	/api/courses{id}/	Update a course
DELETE	/api/courses{id}/	Delete a course

10. Validation and error Handling

Input validation is implemented using Django REST Framework serializers.

The system checks whether the course duration is greater than zero.

For example:

If the user enters:

Duration = 0

the system returns a validation error indicating that the duration must be greater than zero.

The Course ID is also defined as unique, which prevents duplicate Course IDs from being stored in the database.

11. User interface

The application provides a simple and professional user interface.

The main page contains:

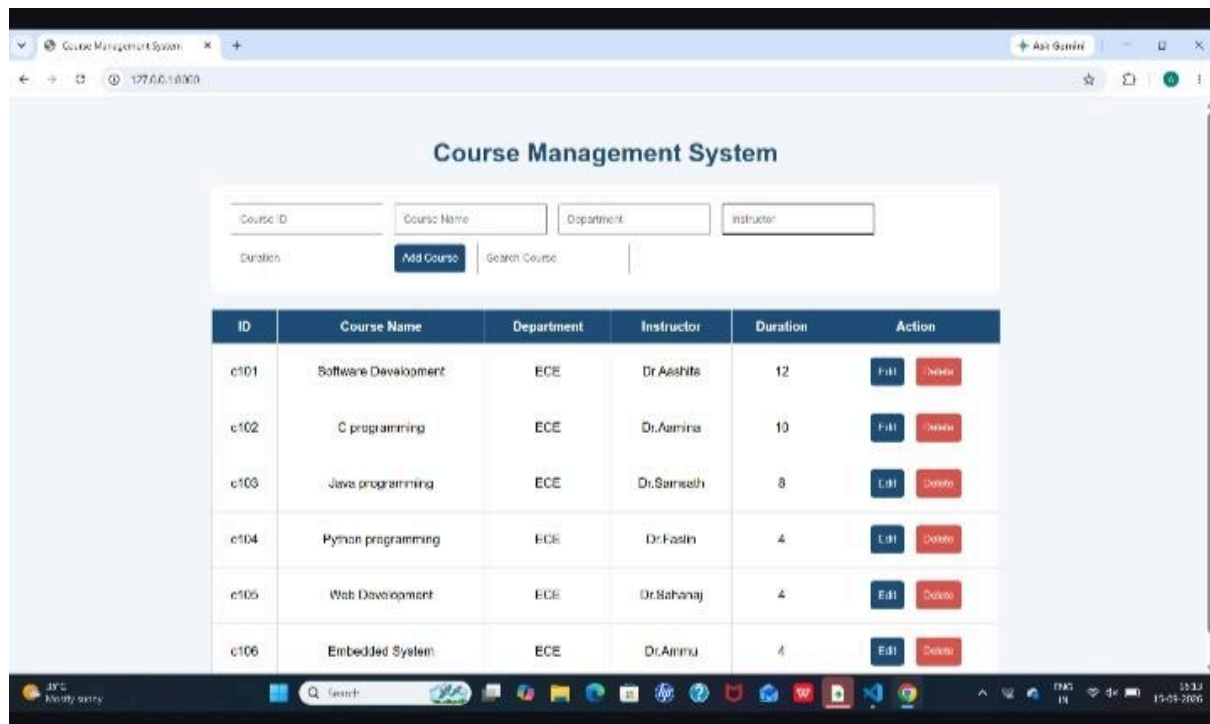
1. Course ID input
2. Course Name input
3. Department input
4. Instructor input
5. Duration input
6. Add Course button
7. Search Course field
8. Course records table
9. Edit button
10. Delete button

The interface is designed using HTML and CSS, while JavaScript is used to communicate with the REST API.

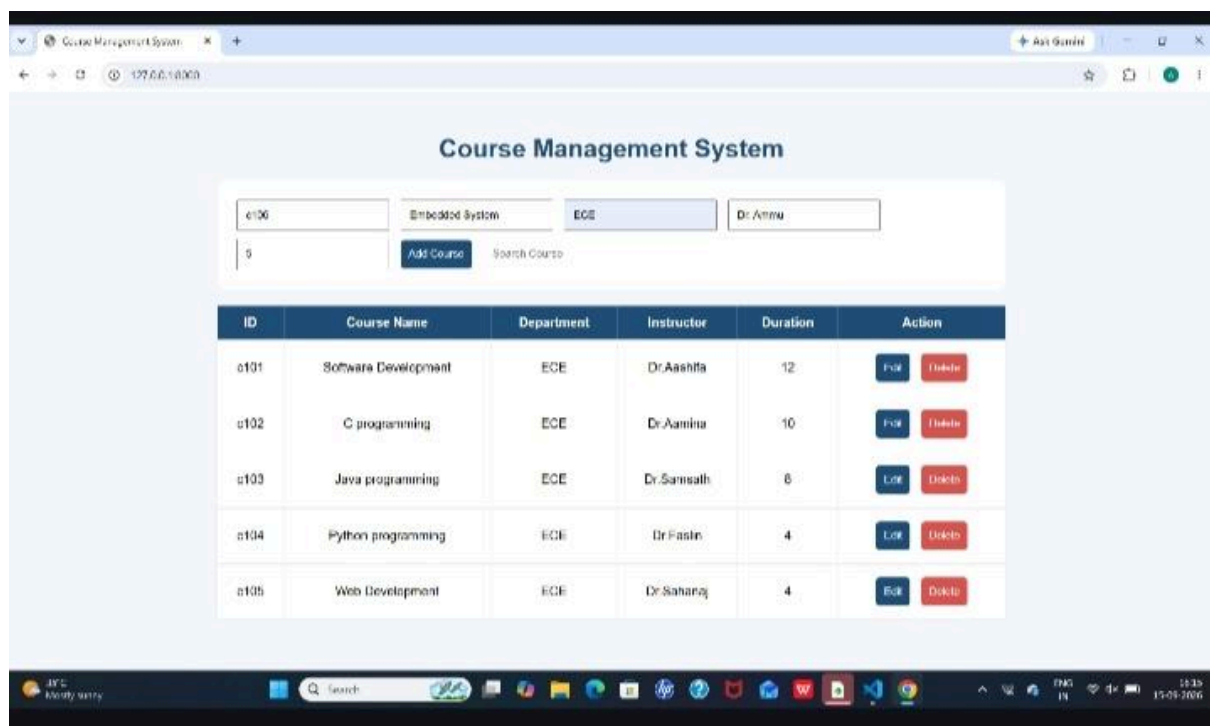
12.Screen Shots

The following screenshots demonstrate the working of the application.

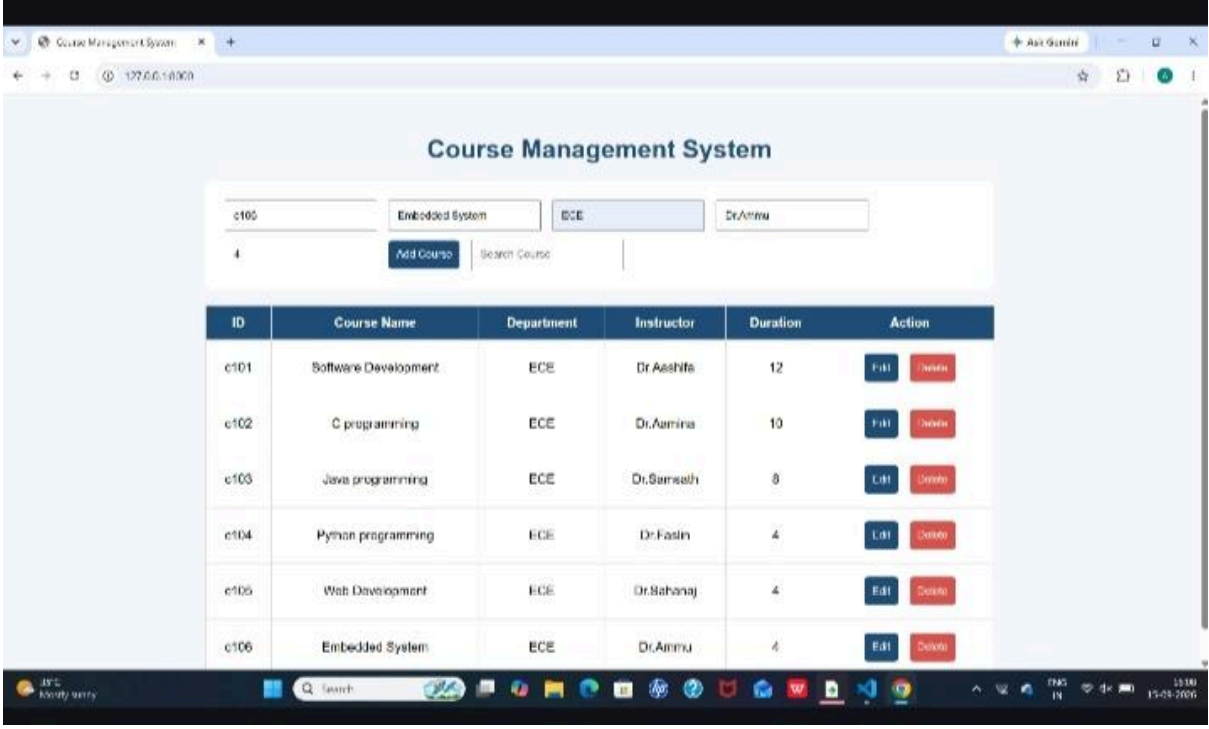
Screenshot 1 – Main Course Management Page



Screenshot 2 – Adding a Course



Screenshot 3 – Course Added Successfully

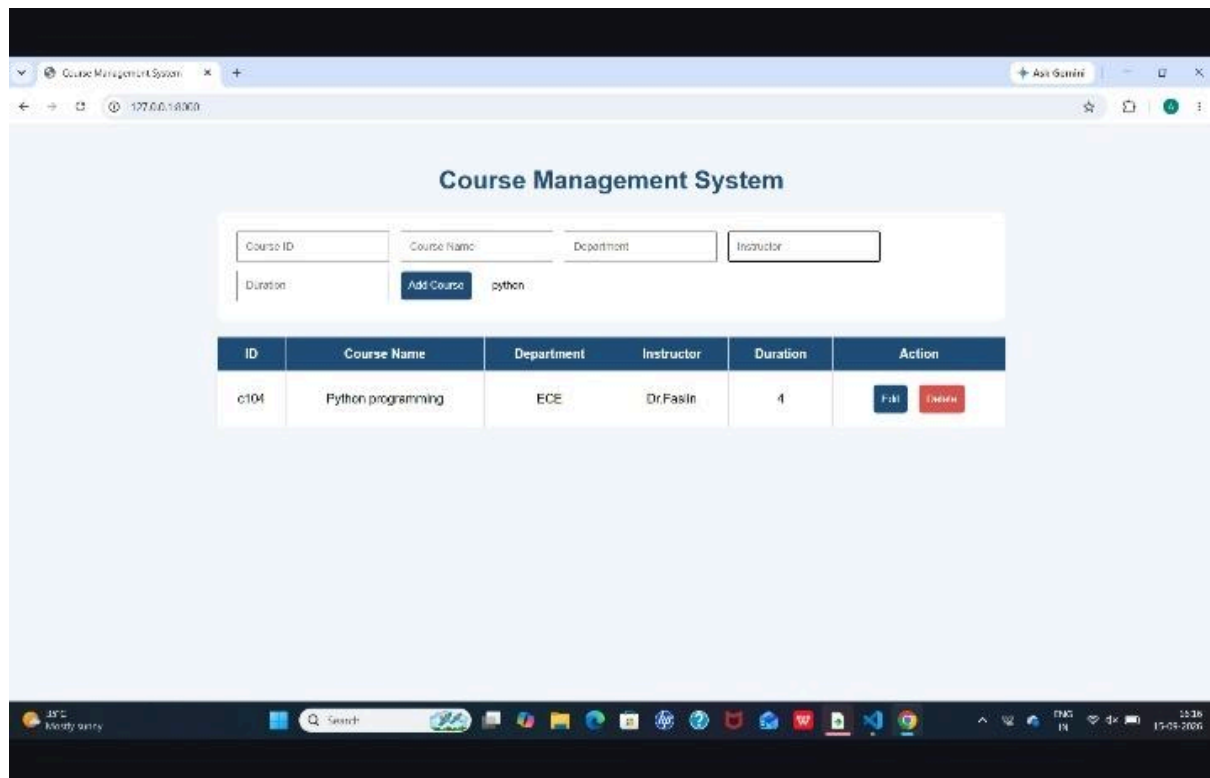


The screenshot displays the 'Course Management System' web application. At the top, the browser address bar shows '127.0.0.1:8000'. The system title 'Course Management System' is centered. Below the title, there is a form with input fields for 'Course ID' (containing 'c105'), 'Course Name' (containing 'Embedded System'), 'Department' (containing 'ECE'), and 'Instructor' (containing 'Dr.Amnu'). A 'Duration' field contains the value '4'. Below these fields are two buttons: 'Add Course' (highlighted in blue) and 'Search Course'. Below the form is a table listing existing courses.

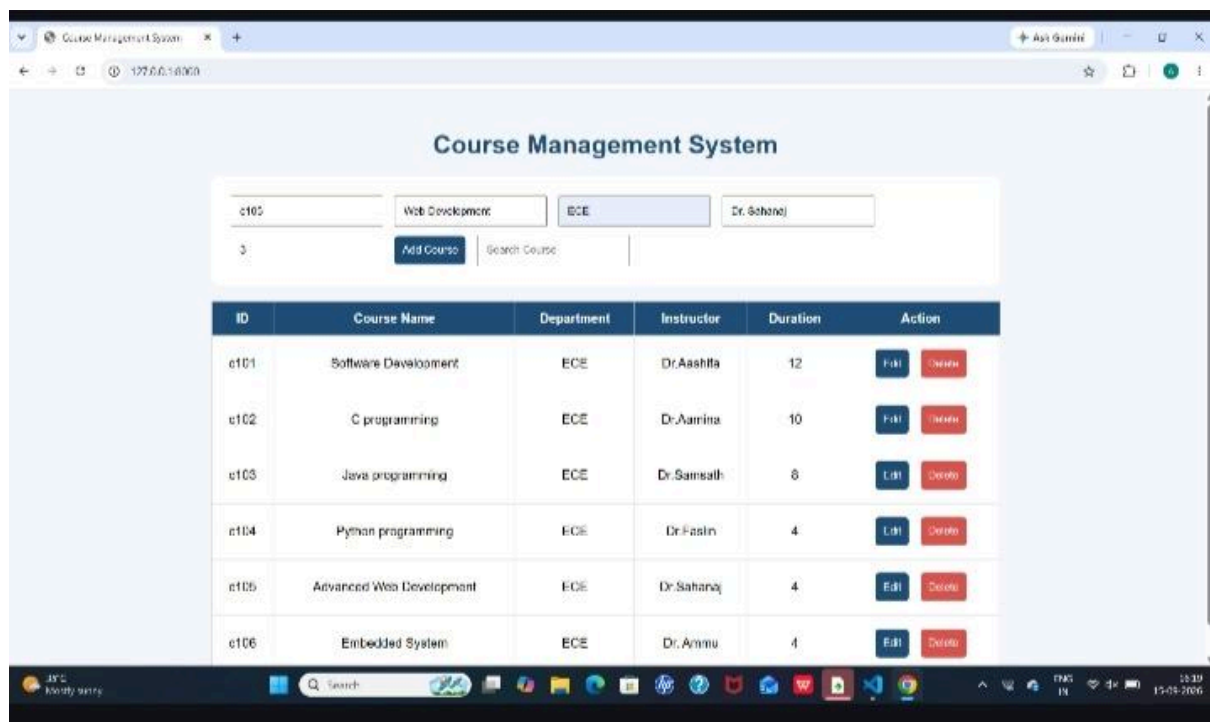
ID	Course Name	Department	Instructor	Duration	Action
c101	Software Development	ECE	Dr.Aashita	12	Edit Delete
c102	C programming	ECE	Dr.Aamira	10	Edit Delete
c103	Java programming	ECE	Dr.Sameer	8	Edit Delete
c104	Python programming	ECE	Dr.Fatin	4	Edit Delete
c105	Web Development	ECE	Dr.Sahana	4	Edit Delete
c106	Embedded System	ECE	Dr.Amnu	4	Edit Delete

The Windows taskbar at the bottom shows the system clock as 11:10 on 15-03-2020.

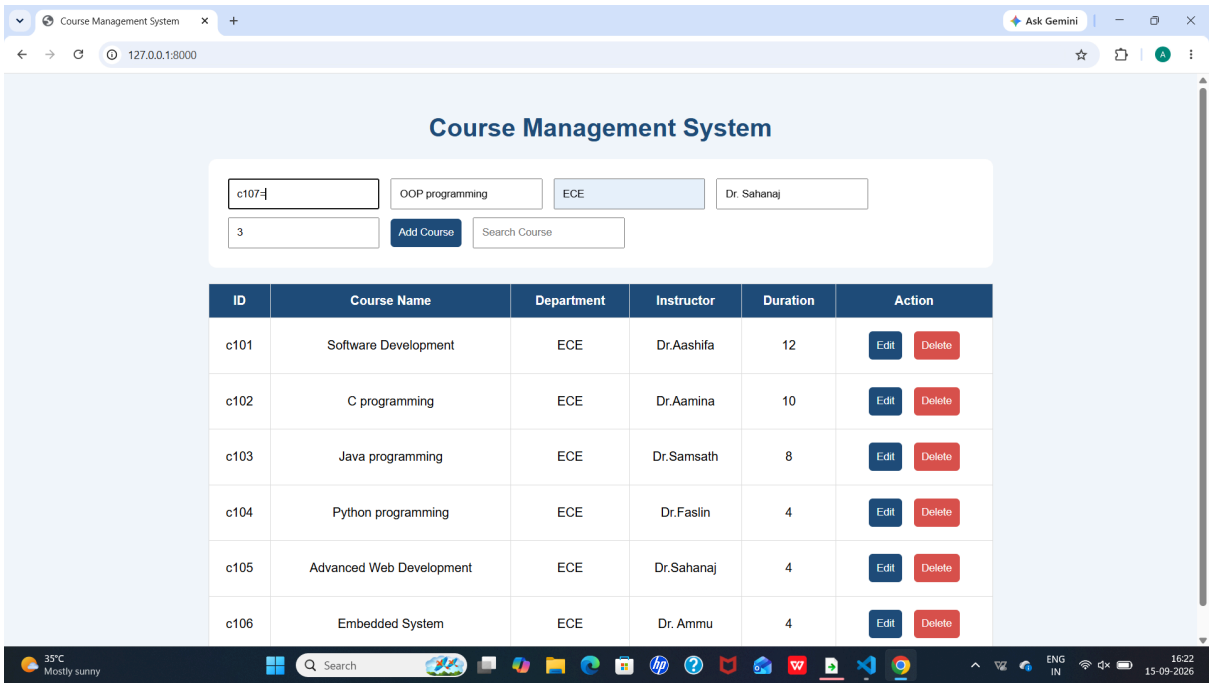
Screenshot 4 – Search Course



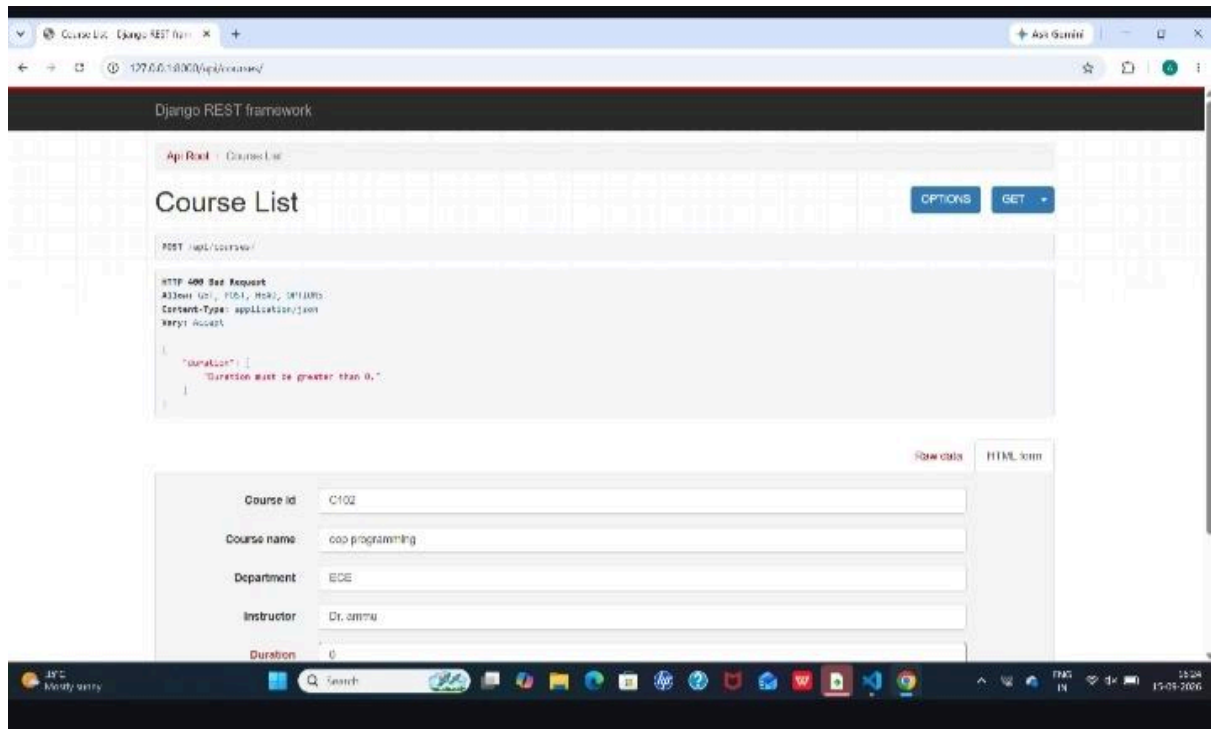
Screenshot 5 – Edit Course



Screenshot 6 – Delete Course



Screenshot 7 – Validation



13. Postman API testing

Postman was used to test the REST APIs of the Course Management System.

GET Request

Method: GET

Endpoint:

<http://127.0.0.1:8000/api/courses/>

Purpose:

To retrieve all course records.

Expected Result:

Status code 200 OK

POST Request

Method: POST

Endpoint:

<http://127.0.0.1:8000/api/courses/>

Sample JSON:

```
{  
  "course_id": "C102",  
  "course_name": "Java Programming",  
  "department": "CSE",  
  "instructor": "Dr. Ravi",  
  "duration": 12  
}
```

Expected Result:

Status code 201 Created

PATCH Request

Method: PATCH

Endpoint:

<http://127.0.0.1:8000/api/courses/10/>

Sample JSON:

```
{  
  "course_name": "Advanced Java Programming"  
}
```

Expected Result:

Status code 200 OK

DELETE Request

Method: DELETE

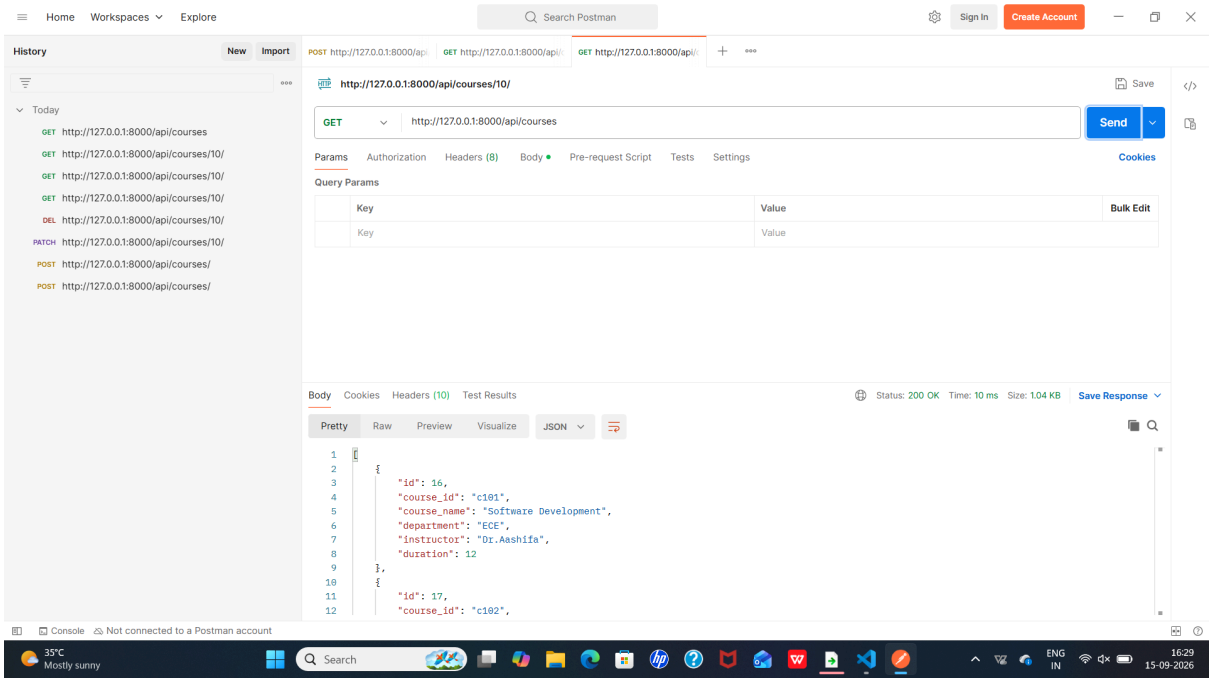
Endpoint:

<http://127.0.0.1:8000/api/courses/10/>

Expected Result:
Status code 204 No Content

Postman Screenshots

GET :



POST:

Home Workspaces Explore Search Postman Sign In Create Account

History New Import

Today

- DEL http://127.0.0.1:8000/api/courses/10/
- PATCH http://127.0.0.1:8000/api/courses/10/
- POST http://127.0.0.1:8000/api/courses/
- POST http://127.0.0.1:8000/api/courses/
- GET http://127.0.0.1:8000/api/courses/

POST http://127.0.0.1:8000/api/courses/

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers Test Results Status: 201 Created Time: 67 ms Size: 454 B Save Response

Pretty Raw Preview Visualize Text

1

35°C Mostly sunny Search 16:27 15-09-2026

PATCH:

Home Workspaces Explore Search Postman Sign In Create Account

History New Import

Today

- DEL http://127.0.0.1:8000/api/courses/10/
- PATCH http://127.0.0.1:8000/api/courses/10/
- POST http://127.0.0.1:8000/api/courses/10/
- POST http://127.0.0.1:8000/api/courses/
- GET http://127.0.0.1:8000/api/courses/

PATCH http://127.0.0.1:8000/api/courses/10/

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

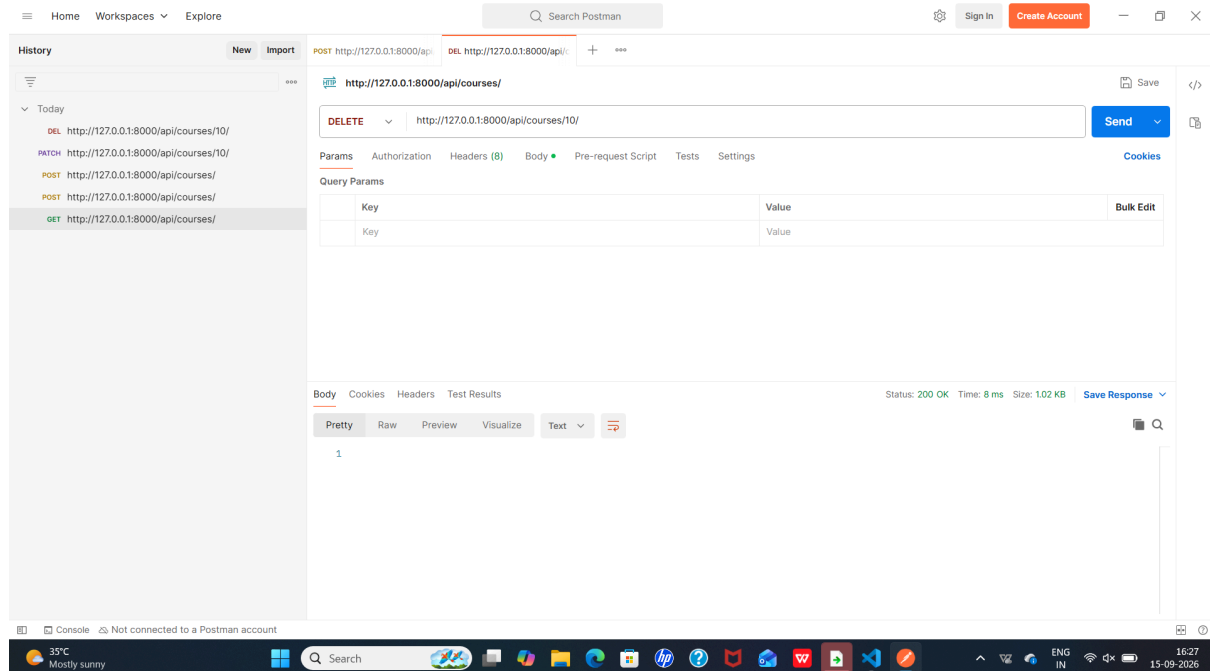
Body Cookies Headers Test Results Status: 200 OK Time: 26 ms Size: 472 B Save Response

Pretty Raw Preview Visualize Text

1

35°C Mostly sunny Search 16:27 15-09-2026

DELETE:



14. Testing Results

The major functionalities of the application were tested successfully.

Test	Operation	Expected Result	Status
1	Add course	Course should be created	Passed
2	View course	Course list should be displayed	Passed
3	Edit course	Course name	Passed

		should be updated	
4	Delete course	Course name should be deleted	Passed
5	Search course	Matching courses should be displayed	Passed
6	Duration visualization	Zero/negative duration should be rejected	Passed
7	GET API	Course data should be returned	Passed
8	POST API	New course should be created	Passed
9	PATCH API	Course should be updated	Passed
10	DELETE API	Course should be deleted	Passed

15.Challenges and Solutions

During development, some challenges were faced while setting up the Django project, configuring the REST API, connecting the frontend with the backend, and testing the APIs.

Challenge 1: Django Project Setup

Setting up the Python virtual environment and Django project was initially challenging.

Solution:

A virtual environment was created and Django was installed using Python. The project was then configured and migrations were performed successfully.

Challenge 2: API Integration

Connecting the JavaScript frontend with the Django REST API required proper API endpoints and JSON data.

Solution:

JavaScript `fetch()` was used to communicate with the REST API.

Challenge 3: Input Validation

Invalid duration values had to be prevented.

Solution:

Validation was implemented in the Django REST Framework serializer.

Challenge 4: API Testing

Different HTTP methods had to be tested.

Solution:

Postman was used to test GET, POST, PATCH, and DELETE requests.

16. Future Enhancements

The following features can be added in future versions:

- User authentication and login.
- Admin dashboard.
- Course filtering by department.
- Advanced search.
- Course enrollment management.
- Course reports and analytics.
- Exporting course information to PDF or Excel.
- Deployment to a cloud server.
- Improved responsive design.
- Role-based access control.

17. Conclusion

The Course Management System successfully provides a simple digital solution for managing course information.

The application implements CRUD operations using Django REST Framework and SQLite. HTML, CSS, and JavaScript are used to provide the frontend interface.

The REST APIs were tested successfully using Postman. Input validation was also implemented to improve data accuracy.

This project provided practical experience in frontend development, Python Django backend development, REST API creation, database management, CRUD operations, API testing, and project documentation.

18. GitHub Repository

The complete source code and project documentation are maintained in the GitHub repository.

GitHub Repository:

[Paste your GitHub repository link here]

Thank you